INSTITUTE OF INFORMATION TECHNOLOGY
JAHANGIRNAGAR UNIVERSITY
Savar, Dhaka-1342

LAB REPORT

Course Code:	ICT-3102
Course Title:	Operating System Lab
Lab No.:	08
Lab Title:	Process Scheduling Algorithms

Submitted To

Mehrin Anannya

Assistant Professor, Institute of Information Technology

Submitted By

Name: Md. Shaon Khan

ID: 1984

Session: 2022-2023

1. Objective

The aim of this lab is to understand and implement three CPU scheduling algorithms: Longest Job First (non-preemptive), Longest Remaining Time First (preemptive), and Round Robin (preemptive). For each algorithm, the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) of every process are calculated, along with the average waiting time and average turnaround time. This lab also gives practice in turning a scheduling algorithm into working code and in observing how each algorithm affects the overall performance of a system.

2. Introduction to the Algorithms

2.1 Longest Job First (LJF) - Non-Preemptive

Longest Job First is a non-preemptive scheduling algorithm. Among all the processes that have already arrived in the ready queue, the process with the largest burst time is chosen to run next. Once a process starts running, it continues until it finishes; it is never interrupted in between. Only the processes that have arrived by the current time are considered while choosing the next process.

Steps of the algorithm:

1. Arrange the processes in increasing order of their arrival time.
2. Among the processes that have arrived by the current time, choose the one with the highest burst time.
3. Run the selected process fully, without any interruption.
4. Repeat the above steps until every process has been executed.

2.2 Longest Remaining Time First (LRTF) - Preemptive

Longest Remaining Time First is the preemptive version of LJF. At every unit of time, the scheduler checks all the arrived processes and runs the one with the largest remaining burst time. If a new process arrives with more remaining time than the process currently running, the CPU is taken away from the running process and given to the new one. This continues until every process is completely finished.

Steps of the algorithm:

1. Arrange the processes in increasing order of their arrival time.
2. At each time unit, select the process with the largest remaining burst time among those that have arrived.
3. Run the selected process for one unit of time and reduce its remaining time by one.
4. Repeat this selection at every time unit, allowing the CPU to switch between processes, until all processes are finished.

2.3 Round Robin (RR) - Preemptive

Round Robin gives each process in the ready queue the CPU for a fixed slice of time called the Time Quantum (TQ), and the processes are handled one after another in order. If a process does not finish within its quantum, the remaining burst time is carried forward and the process is placed back at the end of the queue. If it finishes, it is removed from the queue. Any process that arrives while the CPU is running is added to the ready queue and is handled in the next cycle.

Steps of the algorithm:

1. Arrange the processes in increasing order of their arrival time.
2. Fix a Time Quantum (TQ) for execution.
3. Starting from the first process in the queue, give it the CPU for at most TQ units of time.
4. If the process finishes inside the quantum, remove it from the queue; otherwise subtract the time it has used from its remaining burst time and move it to the end of the queue.
5. Add any newly arrived process to the ready queue.
6. Repeat this until all processes are executed, then calculate the Average Waiting Time and Average Turnaround Time.

3. Source Code and Results

The same set of processes was used to test all three algorithms, so that the results could be compared fairly:

Process	AT	BT	CT	TAT	WT
P1	0	5	-	-	-
P2	1	3	-	-	-
P3	2	8	-	-	-
P4	3	6	-	-	-

3.1 Longest Job First (LJF)

The Python program below implements the non-preemptive LJF scheduling algorithm:

```
n = int(input("Enter number of processes: "))

pid = []
at = []
bt = []
ct = [0] * n
tat = [0] * n
wt = [0] * n
done = [False] * n

avgWT = 0
avgTAT = 0
```

```
for i in range(n):
    pid.append(i + 1)
    arrival, burst = map(int, input(f"Enter Arrival Time and Burst Time for P{i+1}:
").split())
    at.append(arrival)
    bt.append(burst)

completed = 0
current = 0

while completed < n:
    idx = -1
    maxBT = -1

    for i in range(n):
        if not done[i] and at[i] <= current:
            if bt[i] > maxBT:
                maxBT = bt[i]
                idx = i

    if idx == -1:
        current += 1
        continue

    current += bt[idx]
    ct[idx] = current
    tat[idx] = ct[idx] - at[idx]
    wt[idx] = tat[idx] - bt[idx]

    avgWT += wt[idx]
    avgTAT += tat[idx]

    done[idx] = True
    completed += 1

print("\nProcess\tAT\tBT\tCT\tTAT\tWT")
for i in range(n):
    print(f"P{pid[i]}\t{at[i]}\t{bt[i]}\t{ct[i]}\t{tat[i]}\t{wt[i]}")

print(f"\nAverage Waiting Time = {avgWT/n:.2f}")
print(f"Average Turnaround Time = {avgTAT/n:.2f}")
```

Output for Arrival Time = (0, 1, 2, 3) and Burst Time = (5, 3, 8, 6):

Process	AT	BT	CT	TAT	WT
P1	0	5	5	5	0
P2	1	3	22	21	18
P3	2	8	13	11	3
P4	3	6	19	16	10

```
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 0 5
Enter Arrival Time and Burst Time for P2: 1 3
Enter Arrival Time and Burst Time for P3: 2 8
Enter Arrival Time and Burst Time for P4: 3 6

Process AT      BT      CT      TAT     WT
P1      0       5       5       5       0
P2      1       3       22      21      18
P3      2       8       13      11      3
P4      3       6       19      16      10

Average Waiting Time = 7.75
Average Turnaround Time = 13.25
```

Fig 3.1: Console output of the LJF program

Average Waiting Time = 7.75 ms

Average Turnaround Time = 13.25 ms

3.2 Longest Remaining Time First (LRTF)

The Python program below implements the preemptive LRTF scheduling algorithm:

```
n = int(input("Enter number of processes: "))
pid = []
at = []
bt = []
rt = []
ct = [0] * n
tat = [0] * n
wt = [0] * n
avgWT = 0
avgTAT = 0

for i in range(n):
    pid.append(i + 1)
    arrival, burst = map(int, input(f"Enter Arrival Time and Burst Time for P{i+1}:
").split())
    at.append(arrival)
    bt.append(burst)
    rt.append(burst)

completed = 0
current = 0

while completed < n:
    idx = -1
    maxRT = -1
    # find the process with the longest remaining time
    for i in range(n):
        if rt[i] > 0 and at[i] <= current:
            if rt[i] > maxRT:
                maxRT = rt[i]
                idx = i

    # if no process has arrived yet
    if idx == -1:
```

```
        current += 1
        continue

    # run the selected process for 1 unit of time
    rt[idx] -= 1
    current += 1

    # if the process finishes
    if rt[idx] == 0:
        ct[idx] = current
        tat[idx] = ct[idx] - at[idx]
        wt[idx] = tat[idx] - bt[idx]
        avgWT += wt[idx]
        avgTAT += tat[idx]
        completed += 1

print("\nProcess\tAT\tBT\tCT\tTAT\tWT")
for i in range(n):
    print(f"P{pid[i]}\t{at[i]}\t{bt[i]}\t{ct[i]}\t{tat[i]}\t{wt[i]}")

print(f"\nAverage Waiting Time = {avgWT/n:.2f}")
print(f"Average Turnaround Time = {avgTAT/n:.2f}")
```

Output for the same set of processes:

Process	AT	BT	CT	TAT	WT
P1	0	5	19	19	14
P2	1	3	20	19	16
P3	2	8	21	19	11
P4	3	6	22	19	13

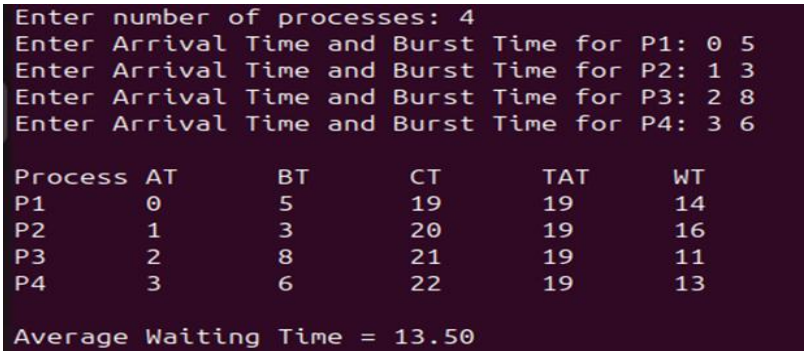

Fig 3.2: Console output of the LRTF program

Average Waiting Time = 13.50 ms

Average Turnaround Time = 19.00 ms

3.3 Round Robin (RR)

The Python program below implements the Round Robin scheduling algorithm, with the time quantum entered by the user:

```
n = int(input("Enter number of processes: "))

pid = []
at = []
bt = []
rt = []
ct = [0] * n
tat = [0] * n
wt = [0] * n

avgWT = 0
avgTAT = 0

for i in range(n):
    pid.append(i + 1)
    arrival, burst = map(int, input(f"Enter Arrival Time and Burst Time for P{i+1}:
").split())
    at.append(arrival)
    bt.append(burst)
    rt.append(burst)

tq = int(input("Enter Time Quantum: "))

completed = 0
current = 0

while completed < n:
    done = True

    for i in range(n):
        if rt[i] > 0 and at[i] <= current:
            done = False

            if rt[i] > tq:
                current += tq
                rt[i] -= tq
            else:
                current += rt[i]
                rt[i] = 0
                ct[i] = current
                tat[i] = ct[i] - at[i]
                wt[i] = tat[i] - bt[i]

                avgWT += wt[i]
                avgTAT += tat[i]
                completed += 1

    if done:
        current += 1  # CPU idle

print("\nProcess\tAT\tBT\tCT\tTAT\tWT")
for i in range(n):
    print(f"P{pid[i]}\t{at[i]}\t{bt[i]}\t{ct[i]}\t{tat[i]}\t{wt[i]}")

print(f"\nAverage Waiting Time = {avgWT/n:.2f}")
print(f"Average Turnaround Time = {avgTAT/n:.2f}")
```

Output for the same set of processes with Time Quantum = 2:

Process	AT	BT	CT	TAT	WT
P1	0	5	16	16	11
P2	1	3	11	10	7
P3	2	8	22	20	12
P4	3	6	20	17	11

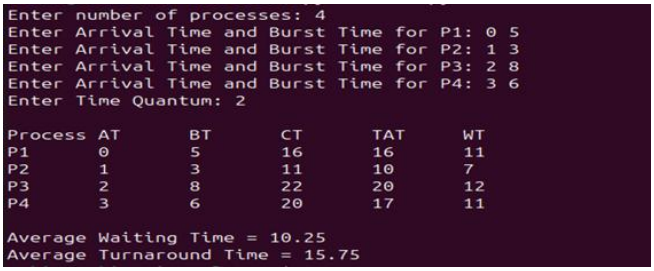

Fig 3.3: Console output of the Round Robin program

Average Waiting Time = 10.25 ms

Average Turnaround Time = 15.75 ms

4. Comparison of Results

The table below shows the average waiting time and average turnaround time obtained from each algorithm, using the same set of processes (AT = 0, 1, 2, 3 and BT = 5, 3, 8, 6):

Algorithm	Avg. Waiting Time (ms)	Avg. Turnaround Time (ms)
LJF (non-preemptive)	7.75	13.25
LRTF (preemptive)	13.50	19.00
Round Robin (TQ = 2)	10.25	15.75

For this set of processes, LJF gives the lowest average waiting time and turnaround time. This happens because there are only four short processes, and in this particular order of arrival, running the longest job first still finishes efficiently. LRTF performs the worst, because it keeps switching to whichever process has the most work left, which makes a shorter job like P2 wait a long time before it finally finishes. Round Robin falls in between the two: since the CPU time is shared fairly using a fixed quantum, no process is starved as it is in LRTF, but there is still some extra waiting time from context switching compared to LJF.

5. Discussion

Longest Job First keeps the number of context switches low and works well when the burst time of each process is known beforehand, but it is unfair to shorter processes, since they may have to wait a long time if longer jobs keep arriving. LRTF, being the preemptive form of LJF, adjusts to newly arriving processes, but it can starve short processes even more, because a newly arrived long process can interrupt the

running process again and again. Round Robin avoids starvation completely, since every process is guaranteed a turn within one cycle of the queue, which makes it well suited for time-sharing systems. However, its performance depends heavily on the chosen time quantum: too small a quantum increases overhead due to frequent context switching, while too large a quantum makes Round Robin behave almost like First Come First Served.

6. Conclusion

In this lab, three CPU scheduling algorithms - Longest Job First, Longest Remaining Time First, and Round Robin - were implemented and tested on the same set of processes. Running each program and comparing the resulting completion time, turnaround time, and waiting time showed how the choice of scheduling algorithm directly affects the waiting time of processes and the overall responsiveness of the system. This lab also highlighted the trade-off between throughput-oriented, non-preemptive strategies such as LJF and fairness-oriented, preemptive strategies such as Round Robin, and explained why real operating systems often prefer quantum-based approaches like Round Robin for interactive, time-shared workloads.